

Практическое занятие 2. Middleware, переменные окружения, статика и тест

Цель: Научиться добавлять middleware для сбора метрик запросов, настраивать конфигурацию приложения через переменные окружения, обеспечивать работу статических файлов в production-режиме (админка со стилями при `DEBUG=False`) и писать простой интеграционный тест.

Продолжительность: 2–3 академических часа.

Пререквизиты: Выполнено занятие 1, создан проект с моделью `Product`, настроена админ-панель, работает валидация.

1. Что будем делать

1. Добавим middleware, который подсчитывает общее количество запросов, количество ответов 2xx, 4xx, 5xx и выводит статистику в консоль (или файл).
2. Вынесем настройки `DEBUG`, `SECRET_KEY` в файл `.env` с помощью `python-dotenv`.
3. Настроим отдачу статических файлов через `whitenoise`, чтобы при `DEBUG=False` админ-панель отображалась со стилями.
4. Напишем интеграционный тест, проверяющий доступность главной страницы (или специального эндпоинта `/ping/`).

Все изменения вносятся в существующий проект, созданный на занятии 1.

2. Middleware для сбора метрик

Теоретическая справка: Middleware – это компонент, который обрабатывает запрос до того, как он попадёт в представление, и ответ – после того, как представление его сформировало. Мы можем перехватывать каждый HTTP-запрос, анализировать статус ответа и накапливать статистику.

2.1. Создание файла `middleware.py`

В папке вашего приложения (например, `products`) создайте файл `middleware.py`:

```
python
```

```
import time
```

```
class MetricsMiddleware:
```

```
    """
```

```
    Middleware для сбора метрик HTTP-запросов.
```

```
    Считает общее количество запросов и количество ответов с кодами 2xx,  
    4xx, 5xx.
```

```
    """
```

```
    def __init__(self, get_response):
```

```
        self.get_response = get_response
```

```
        # Инициализация счётчиков (атрибуты класса)
```

```
        self.total_requests = 0
```

```
        self.status_2xx = 0
```

```
        self.status_4xx = 0
```

```
        self.status_5xx = 0
```

```
        self.request_count_since_last_log = 0
```

```
    def __call__(self, request):
```

```
        # Обработка запроса
```

```
        response = self.get_response(request)
```

```
        # Увеличиваем общий счётчик
```

```
        self.total_requests += 1
```

```
        self.request_count_since_last_log += 1
```

```
        # Определяем категорию статуса ответа
```

```
        status_code = response.status_code
```

```
        if 200 <= status_code < 300:
```

```
            self.status_2xx += 1
```

```
        elif 400 <= status_code < 500:
```

```
            self.status_4xx += 1
```

```
        elif 500 <= status_code < 600:
```

```
            self.status_5xx += 1
```

```
        # Выводим статистику каждые 5 запросов (или каждый раз – по желанию)
```

```
        if self.request_count_since_last_log >= 5:
```

```
            self._log_metrics()
```

```
            self.request_count_since_last_log = 0
```

```
        return response
```

```
def _log_metrics(self):
    """Выводит накопленную статистику в консоль."""
    print("=== METRICS ===")
    print(f"Total requests: {self.total_requests}")
    print(f"2xx: {self.status_2xx}, 4xx: {self.status_4xx}, 5xx: {self.status_5xx}")

    print("=====")
```

Пояснения:

- `__init__` вызывается один раз при запуске приложения, инициализируются счётчики.
- `__call__` вызывается для каждого запроса. Мы получаем ответ, увеличиваем счётчики, и каждые 5 запросов выводим статистику.
- Можно изменить вывод на запись в файл `metrics.log` (задание для самостоятельной работы).

2.2. Подключение middleware в settings.py

Откройте `exam_project/settings.py` и добавьте `products.middleware.MetricsMiddleware` в список `MIDDLEWARE` (лучше в конец, чтобы не конфликтовать с системными middleware):

python

```
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
    'products.middleware.MetricsMiddleware',  # наша middleware
]
```

2.3. Проверка работы middleware

Запустите сервер и откройте в браузере несколько страниц админ-панели (список товаров, добавление, редактирование). В консоли, где запущен сервер, каждые 5 запросов должна появляться статистика.

Если вы хотите видеть статистику после каждого запроса, замените условие `if self.request_count_since_last_log >= 5:` на `if True:` (или просто вызывайте `self._log_metrics()` каждый раз).

3. Конфигурация через переменные окружения (файл .env)

Теоретическая справка: Хранить секретные ключи и настройки прямо в коде (`settings.py`) небезопасно, особенно если вы выкладываете проект в публичный репозиторий. Вместо этого настройки читаются из переменных окружения (файл `.env`). Это также позволяет менять поведение приложения без правки кода.

3.1. Установка python-dotenv

Активируйте виртуальное окружение и установите пакет:

```
bash

pip install python-dotenv
```

3.2. Создание файла .env в корне проекта

В папке, где находится `manage.py`, создайте файл `.env` со следующим содержимым:

```
env

DEBUG=False

SECRET_KEY=ваш_секретный_ключ_придумайте_сложный
```

Важно: Никогда не загружайте `.env` в систему контроля версий (добавьте его в `.gitignore`).

3.3. Настройка settings.py для чтения из .env

В начале файла `settings.py` добавьте код загрузки переменных:

```
python

import os
from dotenv import load_dotenv
```

```
# Загружаем переменные из файла .env
```

```
load_dotenv()
```

```
# Читаем настройки из окружения с значениями по умолчанию
```

```
SECRET_KEY = os.getenv('SECRET_KEY')
```

```
DEBUG = os.getenv('DEBUG', 'False') == 'True'
```

```
# Для учебных целей разрешаем все хосты (в реальном проекте указывайте конкретные)
```

```
ALLOWED_HOSTS = ['*']
```

Пояснение:

- `load_dotenv()` загружает переменные из `.env` в переменные окружения Python.
- `os.getenv('DEBUG', 'False')` – если переменная не задана, по умолчанию `'False'`.
- Сравнение с `'True'` превращает строку в булево значение.

3.4. Проверка работы

Запустите сервер. Если `DEBUG=False`, вы заметите, что страницы ошибок стали менее информативными, а также перестала отдаваться статика (админка «рассыпается»). Это нормально – на следующем шаге настроим статику.

4. Настройка статических файлов при `DEBUG=False`

Теоретическая справка: При `DEBUG=False` Django перестаёт автоматически отдавать статические файлы (CSS, JS, изображения). В production-среде их должен отдавать веб-сервер (nginx) или специальная библиотека. Мы используем `whitenoise` – простой и надёжный способ для учебных проектов.

4.1. Установка whitenoise

```
bash
```

```
pip install whitenoise
```

4.2. Настройка settings.py

В `MIDDLEWARE` добавьте `whitenoise.middleware.WhiteNoiseMiddleware` сразу после `SecurityMiddleware` (до всех остальных `middleware`):

```
python
```

```
MIDDLEWARE = [  
    'django.middleware.security.SecurityMiddleware',  
    'whitenoise.middleware.WhiteNoiseMiddleware',  # добавьте эту строку  
    # ... остальные middleware ...  
]
```

Добавьте настройки для статики (внизу файла):

```
python
```

```
STATIC_URL = '/static/'
```

```
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')
```

4.3. Сбор статики

Выполните команду для сбора всех статических файлов в одну папку:

```
bash
```

```
python manage.py collectstatic
```

Во время выполнения подтвердите действие (нажмите `yes`). После этого появится папка `staticfiles` с копиями CSS и JS админ-панели.

4.4. Проверка работы статики

1. Убедитесь, что в `.env` установлено `DEBUG=False`.
2. Перезапустите сервер.
3. Откройте админ-панель – она должна отображаться со стилями (иконки, оформление).

Если стили не загрузились, проверьте:

- `whitenoise` добавлен в `MIDDLEWARE`;
 - выполнен `collectstatic`;
 - в `settings.py` правильно задан `STATIC_ROOT`.
-

5. Интеграционный тест

Теоретическая справка: Интеграционный тест проверяет, что основные компоненты приложения работают вместе. Мы напишем простой тест, который проверяет доступность главной страницы (или специального эндпоинта `/ping/`). Если тест проходит, значит приложение запускается и обрабатывает запросы.

5.1. Создание тестового эндпоинта (опционально)

Если у вас нет главной страницы (вы используете только админку), создайте простой эндпоинт, возвращающий статус 200. Добавьте в `products/views.py`:

```
python

from django.http import JsonResponse

def health_check(request):

    return JsonResponse({'status': 'ok'})
```

В `products/urls.py` добавьте маршрут:

```
python

urlpatterns = [
    path('', views.product_list, name='product_list'), # если у вас уже
    есть
    path('ping/', views.health_check, name='ping'),
    # другие маршруты...
]
```

5.2. Написание теста

В файле `products/tests.py` напишите:

```
python

from django.test import TestCase, Client

class HealthCheckTest(TestCase):
    def test_ping_endpoint(self):
        client = Client()
        response = client.get('/ping/')
```

```
self.assertEqual(response.status_code, 200)

self.assertEqual(response.json(), {'status': 'ok'})
```

Если вы не создавали отдельный эндпоинт, можно протестировать главную страницу (например, `/admin/` или вашу страницу списка товаров).

5.3. Запуск теста

```
bash

python manage.py test
```

Вы должны увидеть `OK` – тест пройден.

6. Задания для самостоятельной работы

1. Модифицируйте `middleware`, чтобы статистика писалась не в консоль, а в файл `metrics.log`. Используйте модуль `logging` или простое открытие файла в режиме добавления.
 2. Добавьте в `.env` переменную для порта (например, `PORT=8000`) и прочитайте её в `manage.py` (опционально).
 3. Проверьте, что при `DEBUG=False` и невыполненном `collectstatic` админка теряет стили. Выполните `collectstatic` и убедитесь, что стили вернулись.
 4. Добавьте в модель `Product` ещё одно поле (например, `is_active BooleanField`) и создайте миграцию, не нарушив работу приложения.
-

Что должно быть в итоге

- Проект, запускающийся с `DEBUG=False` без ошибок статики.
 - Админ-панель имеет нормальный вид (стили загружаются).
 - В консоль (или файл) выводятся метрики запросов.
 - Есть хотя бы один интеграционный тест, который успешно выполняется.
 - Все настройки (`DEBUG`, `SECRET_KEY`) вынесены в `.env`.
-

Контрольные вопросы

1. Зачем нужен middleware? В каком порядке выполняются middleware в Django?
2. Почему при `DEBUG=False` перестают загружаться статические файлы?
3. Как `whitenoise` решает проблему отдачи статики?
4. Что произойдёт, если забыть выполнить `collectstatic` перед запуском с `DEBUG=False`?
5. Как добавить новую переменную окружения и прочитать её в `settings.py`?
6. В чём разница между unit-тестом и интеграционным тестом?